

ENGINEERING MEMORY

5-Person A–Z Prototype Execution Plan

One GitHub repository • Parallel development • Complete end-to-end prototype

1. Final Prototype

Engineering Memory is an evidence-grounded engineering knowledge system for heavy machinery / hydraulic excavator maintenance. A technician selects a machine, enters symptoms and optional hydraulic conditions, and receives ranked historical cases, likely causes, supporting evidence, recommended inspection and previous successful resolution.

Core flow: Frontend → FastAPI → BGE embedding → FAISS retrieval → diagnosis engine → structured result → Frontend.

- Qwen/local LLM is optional for the first working prototype. Do not let its download block development.
- UCI hydraulic data is a hydraulic test-rig benchmark, not real excavator data. Label it correctly.
- Demo machine/case records are prototype data, not real Tata Hitachi records.
- Diagnosis is decision support, not confirmation of the physical root cause.

2. One GitHub Repository

1. Create one repository named engineering-memory and invite all five teammates.
2. Everyone clones the same repository.
3. Create branches: person1-ai, person2-data, person3-backend, person4-frontend, person5-qa.
4. Never commit venv/, node_modules/, .env, downloaded LLMs or huge raw datasets.
5. Commit small working changes. Push branch → Pull Request → review → merge to main.
6. Before new work: git checkout main → git pull → switch to your branch → merge main.
7. Person 3 owns the API contract, Person 4 consumes it, and Person 5 owns final integration testing.

Repository:

```
engineering-memory/  
  backend/  
  frontend/  
  data/  
  models/  
  src/  
  documents/  
  tests/  
  docs/  
  requirements.txt  
  README.md  
  .gitignore
```

3. Person 1 — AI / Engineering Memory Engine

Mission: Make retrieval and diagnosis work independently of Qwen.

3.1 Verify existing environment

1. Activate the shared Python venv.
2. Verify pandas, numpy, sentence-transformers and faiss-cpu import.
3. Run the existing BGE test.
4. Confirm models/engineering_memory.faiss and models/engineering_memory_metadata.csv exist.
5. Do not rebuild the index unless Person 2 changes the case data.

3.2 Build reusable retrieval

1. Create src/memory.py.
2. Load the FAISS index and metadata CSV.
3. Load BAAI/bge-small-en-v1.5.
4. Implement search(query, machine_id, top_k=3).
5. Embed the query with normalized embeddings.

6. Search FAISS using inner-product similarity.
7. Return structured case records instead of printing them.
8. Each result must include case_id, machine_id, component, symptom, failure_mode, inspection_finding, repair_action, outcome and similarity.

3.3 Machine-aware retrieval

1. Accept machine_id separately from symptoms.
2. Prefer selected-machine history when enough evidence exists.
3. If selected-machine history is insufficient, allow relevant cross-machine evidence and mark its source machine.
4. Never pretend cross-machine evidence is machine-specific.

3.4 Evidence sufficiency

1. Define a transparent prototype similarity threshold.
2. Call the value an evidence/similarity score, not calibrated probability.
3. If evidence is weak, return evidence_sufficiency='insufficient'.
4. Return needed information such as hydraulic pressure, oil temperature, flow condition, noise location and recent maintenance history.
5. Never force a diagnosis.

3.5 Diagnosis engine

1. Create src/diagnosis_engine.py.
2. Input: machine context + symptoms + optional sensor values + retrieved cases.
3. Group retrieved cases by failure mode/component.
4. Rank 1–3 candidate causes using retrieval evidence and transparent rules.
5. For every candidate return cause, component, score, reason and supporting case IDs.
6. Generate recommended inspection from historical inspection findings.
7. Return the previous successful repair from the strongest supporting case.
8. Return evidence_sufficiency and source cases.
9. Keep output deterministic and JSON serializable.

3.6 Optional sensor model

1. If time permits, train a simple Random Forest on processed UCI hydraulic features.
2. Save it under models/.
3. Expose a prediction function separately.
4. Do not block the core prototype if this takes too long.

3.7 Test

1. Test slow boom movement + weak digging force.
2. Test slow operation + high oil temperature.
3. Test slow hydraulic response.
4. Test vague/unknown symptoms.
5. Verify every diagnosis has supporting evidence.
6. Commit only after tests pass.

Deliverables: src/memory.py, src/diagnosis_engine.py, optional sensor_model.py, tests and README instructions.

4. Person 2 — Data + Knowledge Pipeline

Mission: Provide clean, consistent engineering memory and document evidence.

1. Lock machines.csv and maintenance_cases.csv schemas.
2. Verify all machine IDs and case IDs.
3. Expand prototype maintenance cases to about 20–30 consistent demo cases.

4. Cover pump leakage, cooler restriction, filter blockage, valve problems, accumulator pressure, oil contamination, cylinder leakage, pilot pressure and line restriction.
5. Create documents/ with prototype engineering documents such as Hydraulic Troubleshooting Guide, Hydraulic Filter SOP and Cooling System SOP.
6. Clearly label generated/demo documents as prototype data.
7. Create document metadata: document_id, title, type, machine/subsystem, version, date and status.
8. Build a simple ingestion script for TXT/PDF text.
9. Split documents into chunks and preserve metadata.
10. If time permits, create a document-chunk FAISS index.
11. Run data-quality checks for missing IDs, missing causes and duplicate cases.
12. Write data/README.md explaining data sources and limitations.
13. Commit and open a Pull Request.

5. Person 3 — FastAPI Backend

Mission: Expose one stable API for the frontend.

5.1 Setup

1. Create backend/main.py.
2. Install FastAPI and Uvicorn.
3. Add GET /health.
4. Run the server locally.

5.2 Required endpoints

- GET /health — service status.
- GET /machines — machine list.
- GET /machines/{machine_id} — machine details/history.
- POST /diagnose — main workflow.
- POST /memory/search — historical retrieval.
- GET /cases — case explorer.
- GET /cases/{case_id} — case detail.
- GET /benchmark — measured metrics/status.
- GET /analysis/failures — failure-analysis data.

5.3 Diagnose endpoint

1. Create a Pydantic request model: machine_id, symptoms and optional oil_temperature, hydraulic_pressure, flow and vibration.
2. Validate machine ID and symptoms.
3. Call Person 1's diagnosis engine.
4. Return the shared JSON response contract.
5. Handle empty input and unknown machines cleanly.
6. Enable CORS for the frontend.
7. Add basic error handling and logging.
8. Give Person 4 exact request/response examples.

5.4 Final backend check

1. Start API from a clean terminal.
2. Test every endpoint.
3. Test a valid diagnosis.
4. Test insufficient evidence.
5. Ensure paths work from the repository root.

6. Commit requirements.txt and startup instructions.

6. Person 4 — Frontend / UI

Mission: Deliver a premium industrial interface and connect the real Diagnose workflow.

6.1 Shell

1. Create frontend React app.
2. Use top navigation, not a permanent left-page sidebar.
3. Create responsive routing.
4. Header: ENGINEERING MEMORY + system status + primary action.
5. Use Deep Graphite, Steel, Warm Off-White, Industrial Amber, Signal Green and Warning Red.
6. Avoid neon gradients, chatbot styling, excessive glassmorphism and excessive rounded cards.

6.2 Eight pages

- Overview — KPIs, knowledge activity, system health and quick actions.
- Diagnose — machine, symptoms, optional sensors, analysis, causes and evidence.
- Engineering Memory — semantic search, filters and historical cases.
- Machines — profiles, hours, status, maintenance and failure history.
- Evidence — documents, retrieved passages and evidence chain.
- Cases — dense engineering case table and case detail.
- Benchmark — retrieval/diagnosis/system metrics.
- Failure Analysis — failure distribution, components, symptoms, repairs and sensor analysis.

6.3 Make Diagnose real

1. Build machine selector.
2. Build symptom textarea.
3. Build optional hydraulic sensor fields.
4. POST to /diagnose.
5. Show loading/analysis state.
6. Show 1–3 candidate causes.
7. Show evidence score with careful wording.
8. Show supporting case IDs and historical findings.
9. Show recommended inspection and previous resolution.
10. Show a strong INSUFFICIENT EVIDENCE state when returned.
11. Never show fake results when the API fails.

6.4 Connect remaining pages

1. Connect Machines to /machines.
2. Connect Cases to /cases.
3. Connect Memory to /memory/search.
4. Connect Benchmark and Failure Analysis.
5. Use demo document data on Evidence only if ingestion is unfinished, clearly labelled.
6. Add loading, empty and error states.

7. Person 5 — Benchmark + QA + Demo

Mission: Prove the prototype works and make the final demo reliable.

1. Create 10–20 diagnosis test queries with expected failure modes.
2. Include pump, cooler, filter, valve, accumulator, oil, cylinder and pilot cases.
3. Measure Recall@1, Recall@3 and MRR.

4. Measure Top-1/Top-3 diagnosis accuracy where ground truth exists.
5. Measure retrieval latency and end-to-end response time.
6. Never invent metrics; show 'Awaiting benchmark' if not measured.
7. Test unknown machine, empty symptom, weak evidence and normal-operation inputs.
8. Build failure-analysis aggregations.
9. Run the complete demo flow from clean startup.
10. Prepare a second demo showing insufficient evidence.
11. Check all pages, API errors, broken navigation and inconsistent IDs.
12. Open bugs and coordinate fixes with the relevant teammate.
13. After fixes, run the complete regression test and record final measured metrics.

8. Shared JSON Response Contract

```
{
  "machine_id": "EXC-001",
  "evidence_sufficiency": "sufficient",
  "likely_causes": [
    {"cause": "Internal pump leakage", "component": "Hydraulic Pump",
     "score": 0.82, "reason": "..."}
  ],
  "historical_evidence": [
    {"case_id": "CASE-001", "similarity": 0.91, "summary": "..."}
  ],
  "recommended_inspection": [...],
  "previous_successful_resolution": "...",
  "source_documents": []
}
```

For insufficient evidence, return evidence_sufficiency='insufficient' and explain what additional information is needed.

9. Git Commands

```
Owner:
git init
git add .
git commit -m "Initial Engineering Memory prototype"
git branch -M main
git remote add origin <GITHUB_URL>
git push -u origin main
```

```
Teammate:
git clone <GITHUB_URL>
cd engineering-memory
git checkout -b personX-module
git add .
git commit -m "Implement <module>"
git push -u origin personX-module
```

```
Update:
git checkout main
git pull origin main
git checkout personX-module
git merge main
```

.gitignore must include: venv/, __pycache__/, .env, node_modules/, .pytest_cache/, *.pyc and downloaded LLM/model files.

10. Integration Order

1. Person 2 locks and commits data.
2. Person 1 makes retrieval + diagnosis reusable.
3. Person 3 exposes them through FastAPI.
4. Person 4 connects Diagnose first, then the remaining pages.
5. Person 5 runs tests while integration happens.
6. All five merge only working changes to main.
7. Run the full project from a clean clone.
8. Only after Diagnose works end-to-end, spend remaining time on visual polish and secondary pages.

11. Definition of Done

- One GitHub repo contains the complete prototype.
- Frontend and FastAPI start successfully.
- Machine list/details work.
- Diagnose accepts a real symptom query.
- BGE + FAISS retrieval works.
- Diagnosis returns ranked causes with evidence.
- Insufficient evidence is handled.
- Memory, Cases, Machines, Evidence, Benchmark and Failure Analysis pages work.
- At least 10 diagnosis tests pass.
- Measured metrics are displayed; no invented numbers.
- README explains setup and run commands.
- Qwen is optional and does not block the prototype.
- Final demo works from a clean clone.

12. Today's Priority

1. Create GitHub repo and invite all five.
2. Person 2 locks data.
3. Person 1 finishes reusable AI engine.
4. Person 3 builds API around it.
5. Person 4 builds UI and connects Diagnose.
6. Person 5 builds tests/benchmark in parallel.
7. Do NOT wait for Qwen.
8. Integrate Diagnose first.
9. Finish secondary pages and polish after the core flow works.
10. Run the final clean-clone demo.